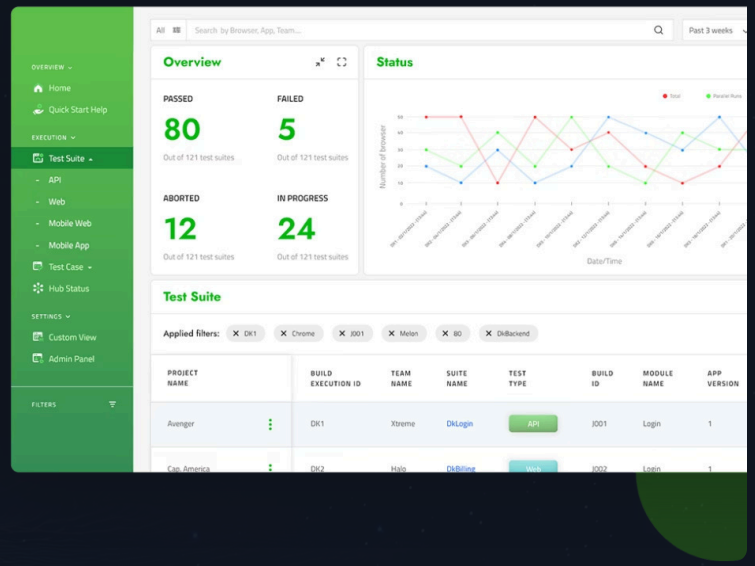


Lab 6: BDD, API Testing, and CI/CD

A Practical Guide to Gherkin, Behave, and Jenkins

Made with GAMMA

Lab Objectives

Today, we will cover the essential tools and techniques for modern test automation. By the end of this lab, you will understand and be able to implement:

Gherkin Syntax

- `Scenario Outline` for data-driven testing
- The `Background` keyword to reduce repetition
- `Tags` for organizing and selectively running tests

Test Automation Framework

- Creating a `Custom Runner` for programmatic control over test execution
- Generating professional test `Reports` with Allure

API Testing & CI/CD

- Writing BDD tests for APIs using Behave and Python
- Automating our test execution with Jenkins

Made with GAMMA

Part 1: Gherkin Fundamentals

Let's start by exploring how to write clean, efficient, and maintainable feature files using advanced Gherkin syntax.

The Problem: Repetitive Scenarios

Often, we need to test the same scenario with different data. For example, testing a login form with both valid and invalid credentials. Writing a separate scenario for each test case leads to code duplication.

Example:

```
Scenario: Validate Successful Login
When I enter a username "webdriver"
And I enter a password "webdriver123"
And I click on the login button
Then I should be presented with the successful login message

Scenario: Validate Unsuccessful Login
When I enter a username "webdriver"
And I enter a password "webdriver1"
And I click on the login button
Then I should be presented with the unsuccessful login message
```

The structure is identical; only the password and the expected message change.

Made with GAMMA

The Solution: Scenario Outline

Gherkin's Scenario Outline allows you to run the same scenario multiple times with different data sets.

Use placeholders

Use placeholders (e.g., <header>) in the scenario steps.

Provide the data

Provide the data in an Examples table.

```
Scenario Outline: Validate - Successful & Unsuccessful Login
When I enter a username "<username>"
And I enter a password "<password>"
And I click on the login button
Then I should be presented with message "<loginValidationText>"

Examples:
| username  | password      | loginValidationText    |
| webdriver | webdriver123  | validation succeeded   |
| webdriver | webdriver1    | validation failed      |
```

This approach is cleaner, more scalable, and easier to maintain.

Made with GAMMA

The Problem: Repetitive Setup Steps

Sometimes, every scenario in a feature file requires the same initial steps (Given steps).

Example:

```
Feature: WebDriver University - Login Page

Scenario: Validate Successful Login
Given I access the webdriver university login page
And I enter a username "webdriver"
When I enter a password "webdriver123"
And I click on the login button
Then I should be presented with the successful login message

Scenario: Validate Unsuccessful Login
Given I access the webdriver university login page
And I enter a username "webdriver"
When I enter a password "webdriver1"
And I click on the login button
Then I should be presented with the unsuccessful login message
```

The two Given steps are repeated in every scenario.

Made with GAMMA

The Solution: The Background Keyword

The Background keyword lets you define a set of steps that will run **before each scenario** in the feature file.

```
Feature: WebDriver University - Login Page

Background:
Given I access the webdriver university login page
And I enter a username "webdriver"

Scenario: Validate Successful Login
When I enter a password "webdriver123"
And I click on the login button
Then I should be presented with the successful login message

Scenario: Validate Unsuccessful Login
When I enter a password "webdriver1"
And I click on the login button
Then I should be presented with the unsuccessful login message
```

This keeps your scenarios focused on their unique actions and validations.

Made with GAMMA

Organizing Tests with Tags

Tags are markers (e.g., @sometag) used to categorize your features and scenarios. This is incredibly useful for organizing your test suite.

Why use tags?

Categorization

Group tests by functionality (@login, @search)

Test Suites

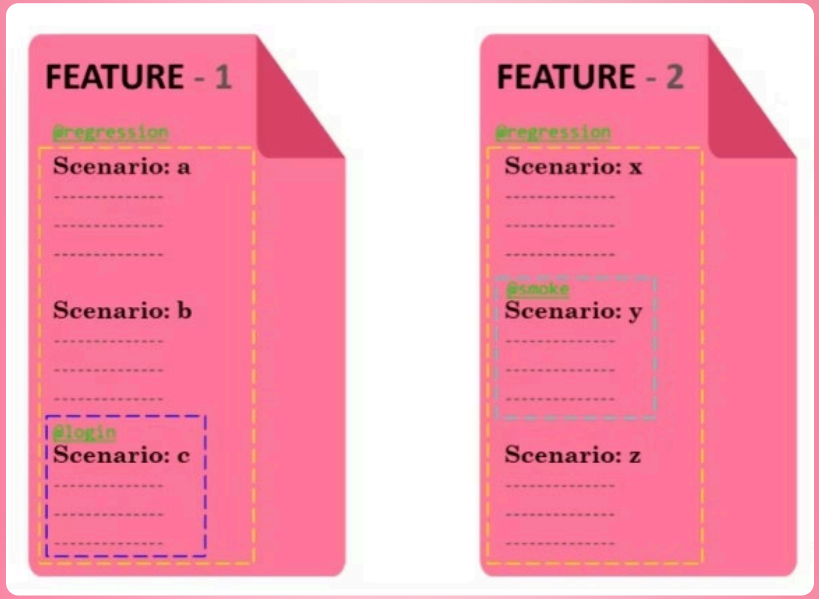
Create different test suites (@smoke, @regression)

Execution Control

Run only a specific subset of tests

You can place tags above Feature or Scenario. A tag on a feature applies to all scenarios within it.

Made with GAMMA



Tagging Strategy Example

Here is a visual representation of how you can apply different tags to your features and scenarios.

Made with GAMMA

Running Tests Using Tags

You can filter which tests to run from the command line by specifying tags.

```
# Run all tests marked as regression tests
python -m behave --tags=@regression

# Run only the smoke tests
python -m behave --tags=@smoke

# Run only the login tests
python -m behave --tags=@login
```

You can also use more complex logic:

- `--tags="not @smoke"` (run everything except smoke tests)
- `--tags="@regression and @login"` (run scenarios that have both tags)

Made with GAMMA

Part 2: Building a Test Automation Framework

The Need for a Custom Runner

While running tests from the command line is easy, a custom runner script gives you powerful programmatic control.

Why use a custom runner?

1 Pre-test Setup

Clean directories, create folders, or start services before tests run

2 Post-test Teardown

Clean up resources, stop services, or generate reports after tests finish

3 Dynamic Configuration

Pass complex configurations or secrets to your tests in a controlled way

Creating a Basic Custom Runner

Here is a simple Python script that executes our Behave tests. It imports behave_main and passes command-line arguments as a list.

custom_runner.py

```
from behave.__main__ import main as behave_main

if __name__ == "__main__":
    # Define arguments programmatically
    behave_args = [
        "--tags", "@regression",
        # You can add more behave arguments here
    ]
    behave_main(behave_args)
```

You run this script just like any other Python file: python custom_runner.py

Made with GAMMA

An Advanced Custom Runner

Let's enhance our runner to perform setup and teardown tasks, like cleaning report directories before a test run.

custom_runner.py

```
import os
import shutil
from behave.__main__ import main as behave_main

def clean_and_create_folder(folder_path):
    """Deletes a folder if it exists and recreates it."""
    if os.path.exists(folder_path):
        shutil.rmtree(folder_path)
    os.makedirs(folder_path, exist_ok=True)

if __name__ == "__main__":
# Define paths
allure_results_folder = "reports/allure-results"
screenshot_folder = "screenshots"

# 1. Perform pre-test setup
print("Cleaning up old reports and screenshots...")
clean_and_create_folder(allure_results_folder)
clean_and_create_folder(screenshot_folder)

# 2. Define Behave arguments and run tests
behave_args = [
    "--tags", "@regression",
    "--format", "allure_behave.formatter:AllureFormatter",
    "--outfile", "reports/allure-results"
]
behave_main(behave_args)

# 3. Perform post-test actions (e.g., generate report)
print("Test execution finished. To see the report, run: allure serve reports/allure-results")
```

Made with GAMMA

Generating Reports with Allure

Allure is an open-source framework that creates beautiful, interactive test reports.

Installation:

1. Install the Python adapter: `pip install allure-behave`
2. Install the Allure command-line tool. Visit the official documentation for instructions for your OS: https://allurereport.org/docs/install/

Made with GAMMA

Integrating Allure with the Custom Runner

To generate Allure reports, you need to tell Behave to use the Allure formatter. We already did this in our advanced runner script:

```
# ... inside if __name__ == "__main__":
    behave_args = [
        "--tags", "@regression",
        # Use the Allure formatter
        "--format", "allure_behave.formatter:AllureFormatter",
        # Specify the output directory for the JSON results
        "--outfile", "reports/allure-results"
    ]
    behave_main(behave_args)
```

When you run this, Behave will generate a set of JSON files in the `reports/allure-results` directory.

Made with GAMMA

Generating and Viewing the Allure Report

The JSON files are not the final report. You need to use the Allure command-line tool to convert them into an HTML report.



Generate the report:

This command processes the JSON files and creates the HTML report in a new directory.

```
allure generate reports/allure-results -o reports/allure-report --clean
```



Open the report:

This command starts a local web server to view the report.

```
allure serve reports/allure-results
```

Made with GAMMA

Adding Screenshots to Reports on Failure

One of the most powerful features of reporting is attaching a screenshot when a UI test fails. This can be done using a Behave hook in the `environment.py` file.

features/environment.py

```
import allure
from allure_commons.types import AttachmentType

def after_scenario(context, scenario):
    # Check if the scenario failed
    if scenario.status == "failed":
        try:
            # Capture screenshot
            screenshot_path = f"screenshots/{scenario.name.replace(' ', '_')}.png"
            context.page.screenshot(path=screenshot_path)

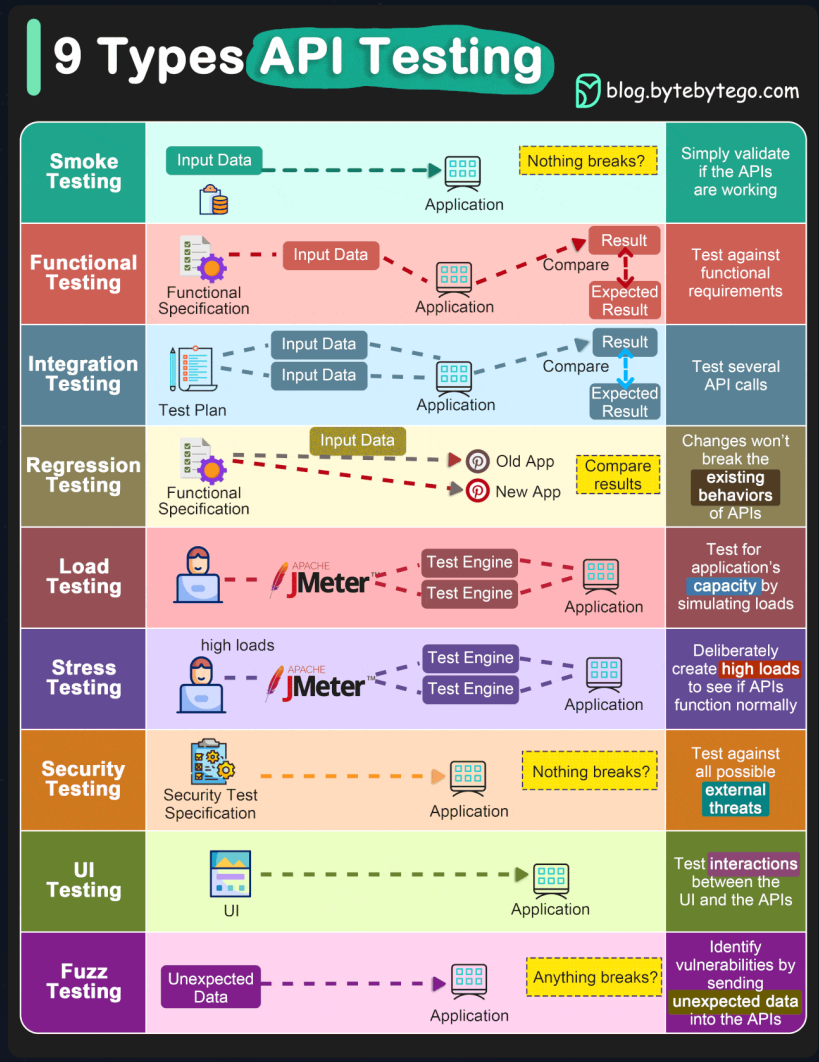
            # Attach screenshot to Allure report
            with open(screenshot_path, "rb") as image_file:
                allure.attach(
                    image_file.read(),
                    name=scenario.name,
                    attachment_type=AttachmentType.PNG
                )
        except Exception as e:
            print(f"Failed to take screenshot: {e}")

    # Don't forget to close your browser page/context if you are using Playwright/Selenium
    if hasattr(context, 'page'):
        context.page.close()
```

Made with GAMMA

Part 3: API Testing with Behave

BDD is not just for UI testing. We can write clear, human-readable tests for our APIs as well.



Made with GAMMA

API Testing Example: Feature File

Here is a Gherkin feature file for testing an API endpoint. The language is descriptive and focuses on the behavior, not the implementation.

check_api_status.feature

```
Feature: Check API status

  Scenario: Check API status
    Given the base URL is "https://goal-tracker-api.onrender.com/api/v1/"
    When I send a GET request to "status"
    Then the response should contain "OPERATIONAL"
    And the response status code should be 200
```

Made with GAMMA

API Testing Example: Step Definitions

The step definitions use the `requests` library to perform the HTTP calls and assertions.

api_steps.py

```
from behave import given, when, then
import requests

@given('the base URL is "{base_url}"')
def step_impl(context, base_url):
    context.base_url = base_url

@when('I send a GET request to "{endpoint}"')
def step_impl(context, endpoint):
    full_url = f"{context.base_url}{endpoint}"
    # Store the response in the context to use in the 'then' step
    context.response = requests.get(full_url)

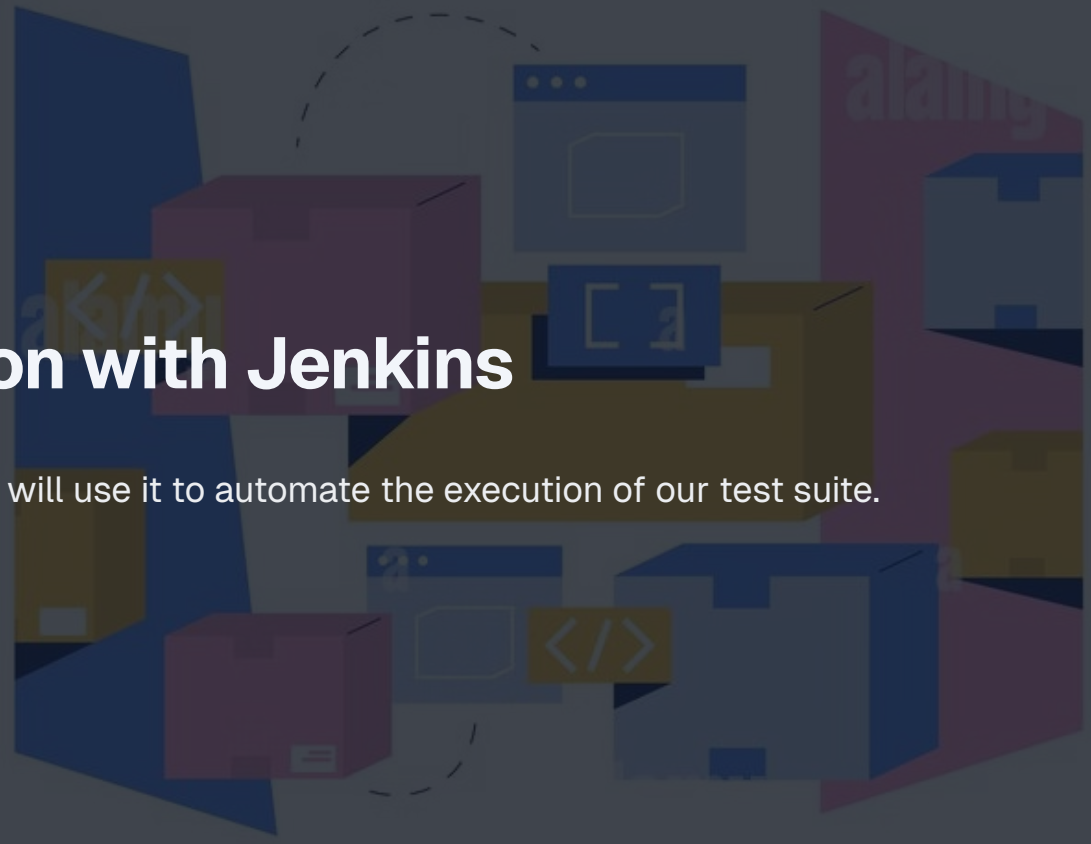
@then('the response should contain "{expected_text}"')
def step_impl(context, expected_text):
    # Assert that the expected text is in the response body
    assert expected_text in context.response.text, \
        f"Response text did not contain '{expected_text}'. Actual text: '{context.response.text}'"

@then('the response status code should be {status_code:d}')
def step_impl(context, status_code):
    # Assert the HTTP status code
    assert context.response.status_code == status_code, \
        f"Expected status code {status_code} but got {context.response.status_code}"
```

Made with GAMMA

Part 4: Continuous Integration with Jenkins

Jenkins is the leading open-source automation server. We will use it to automate the execution of our test suite.



Made with GAMMA

Jenkins Setup: Step-by-Step Guide

1

Installation

Follow the official documentation to install Jenkins on your operating system:
https://www.jenkins.io/doc/book/installing/

2

Create Your First Job

- On the Jenkins dashboard, click **"New Item"**.
- Enter an item name (e.g., `My-Behave-Tests`).
- Select **"Freestyle project"** and click **"OK"**.

Made with GAMMA

Jenkins Setup: Configuring the Job

3. Configure a Build Step

- In the job configuration page, scroll down to the **"Build Steps"** section.
- Click **"Add build step"** and choose:
  - `Execute Windows batch command` (for Windows)
  - `Execute shell` (for macOS/Linux)
- In the command box, enter a simple command to verify your setup, like:

```
python --version
```

Made with GAMMA

Jenkins Setup: Running and Verifying

4. Save and Run the Build

- Click **"Save"**.
- You will be taken to the job's main page. Click **"Build Now"** in the sidebar.
- A new build will appear in the "Build History". Click on it.

5. Check the Console Output

- In the build's page, click **"Console Output"**.
- You should see the output of your command (e.g., `Python 3.9.1`). This confirms Jenkins can execute commands in your environment.

Made with GAMMA

Running Behave Tests in Jenkins

01

Set the Custom Workspace

For Jenkins to find your project files (feature files, step definitions), you must tell it where your project is located.

- Go back to your job's **"Configure"** page.
- Under the "General" tab, click **"Advanced..."**.
- Check the box for **"Use custom workspace"** and enter the full path to your project's root directory (e.g., `C:\Users\YourUser\Documents\my-behave-project`).

02

Update the Build Step

- Scroll down to your build step and change the command to run your tests.

```
python -m behave
```

- Save and run the build again. Check the console output to see your test results.

Made with GAMMA

Parameterizing Jenkins Jobs

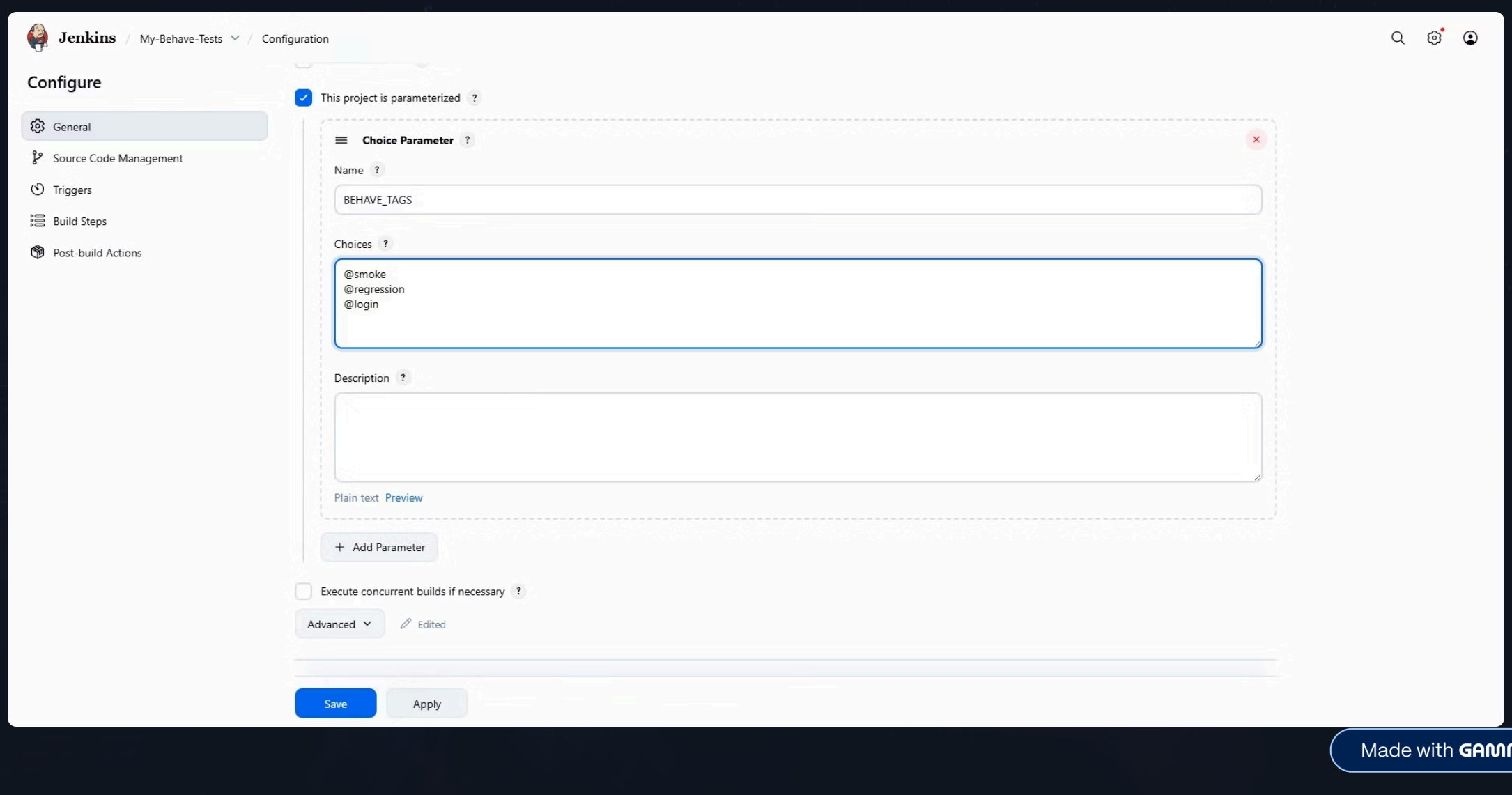
Hardcoding commands is inflexible. Let's parameterize our job so we can choose which tags to run from the Jenkins UI.

8. Add a Choice Parameter

- In the job's **"Configure"** page, under the "General" tab, check the box **"This project is parameterised"**.
- Click **"Add Parameter"** and select **"Choice Parameter"**.
- Configure it as follows:
  - **Name:** `BEHAVE_TAGS`
  - **Choices:** (one per line)

```
@smoke
@regression
@login
```

  - **Description:** `Select the Behave tag to run.`



Made with GAMMA

Using Parameters in Your Build

Now, when you click "Build Now", Jenkins will prompt you to choose a tag. We need to pass this choice to our test runner.

1

Update the Custom Runner to Accept Parameters

Modify `custom_runner.py` to read the tag from an environment variable (set by Jenkins) or a command-line argument.

```
# custom_runner.py
import os
import argparse
# ... (other imports)

if __name__ == "__main__":
    # ... (folder cleanup code)

    # Set up command-line argument parsing
    parser = argparse.ArgumentParser()
    parser.add_argument("--tags", help="Behave tags to
filter scenarios", default="")
    args = parser.parse_args()

    # Get tags from Jenkins environment variable OR
command-line argument
    tags_to_run = os.getenv("BEHAVE_TAGS", args.tags)

    behave_args = [
        f"--tags={tags_to_run}",
        # ... (other args like formatter)
    ]
    behave_main(behave_args)
```

2

Update the Jenkins Build Step

Change the command to execute the custom runner and pass the Jenkins parameter.

- For Windows:

```
python custom_runner.py --tags "%BEHAVE_TAGS%"
```

- For macOS/Linux:

```
python custom_runner.py --tags "$BEHAVE_TAGS"
```

Made with GAMMA

Integrating Allure Reports in Jenkins

Let's configure Jenkins to automatically generate and display our Allure reports.

1

Install the Allure Plugin

- Go to **Manage Jenkins > Plugins**.
- In the **"Available plugins"** tab, search for `Allure` and install it.

2

Configure the Allure Tool

- Go to **Manage Jenkins > Tools**.
- Scroll down to **"Allure Commandline"** and click **"Add Allure Commandline"**.
- Give it a **Name** (e.g., `Allure-2.27.0`).
- Check **"Install automatically"** and select a version from the dropdown.
- Click **"Save"**.



Made with GAMMA

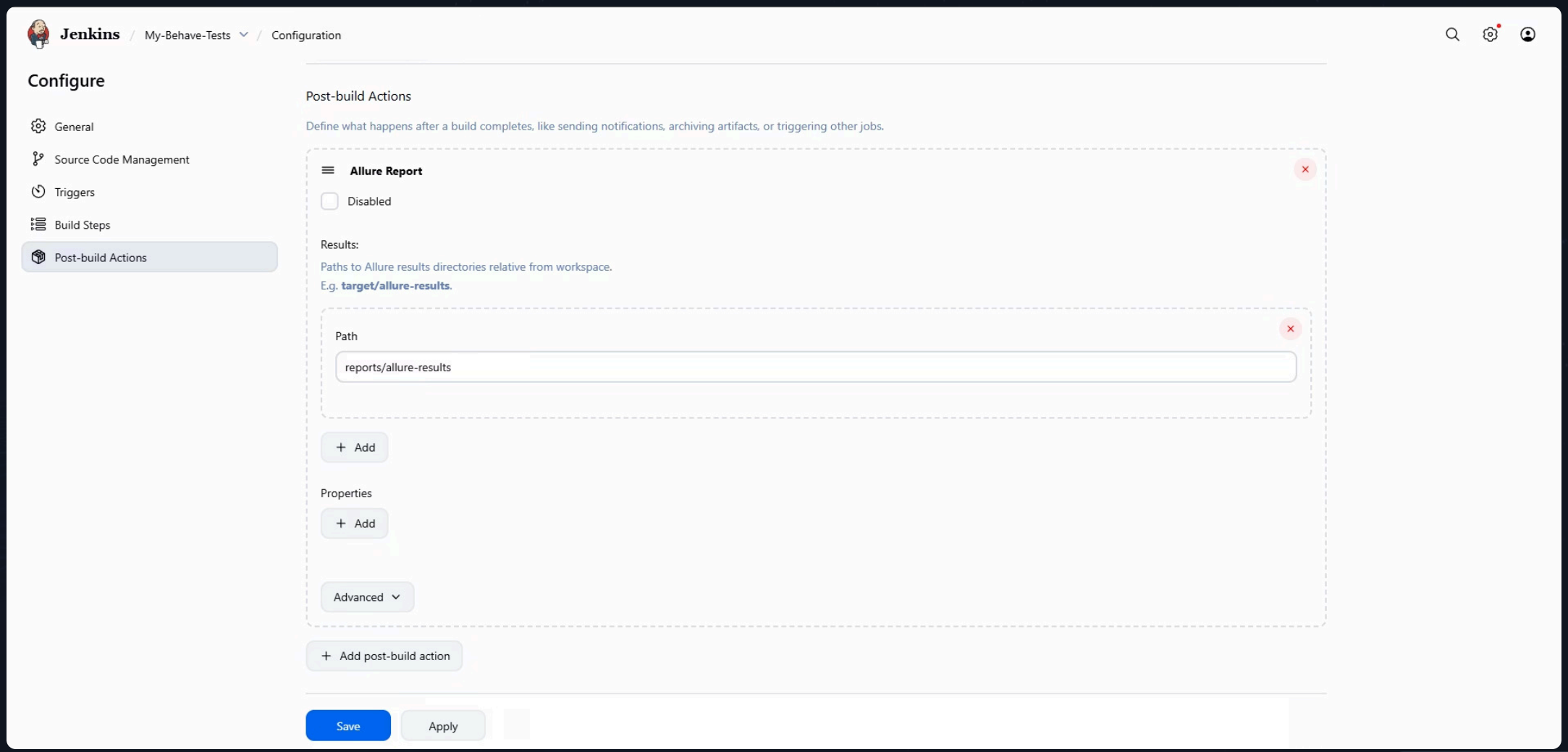
Displaying the Allure Report

13. Add a Post-Build Action

The final step is to tell your job to generate the report after the build is complete.

- Go to your job's **"Configure"** page.
- Scroll to the bottom and click **"Add post-build action"**.
- Select **"Allure Report"**.
- In the **"Results"** path field, enter the directory where Behave saves the JSON files: `reports/allure-results`.
- Click **"Save"**.

Now, after you run a build, a new **"Allure Report"** link will appear in the sidebar of your job's page, taking you to the interactive report.



Made with GAMMA

Lab Requirement

Now it's your turn to apply everything we've learned.

Create API BDD Scenarios:

- Write test scenarios for the public API of **JSONBin** (**https://jsonbin.io/**). Focus on creating, reading, and deleting bins.
- You **must** use `Scenario Outline` and `Background` where appropriate.

Organize Your Tests:

- Use tags to categorize your tests. Specifically, add `@tag1` to half of your tests and `@tag2` to the other half.

Implement the Framework:

- Implement all the necessary step definitions in Python.
- Create a `custom_runner.py` script.
- Configure the runner to generate an **Allure report** and attach screenshots on failure (even for API tests, you can create a placeholder image to practice the attachment logic).

Automate with Jenkins:

- Create a Jenkins job that runs your test suite.
- The job must have **automatic Allure report generation**.
- The job must be **parameterized** with a choice for the tags (`@tag1`, `@tag2`).
- The job must use your `custom_runner.py` to execute the tests.
- Run the job once for `@tag1` and once for `@tag2`.

Made with GAMMA

Thank You

Good luck, and feel free to ask questions

Made with GAMMA